

हाय एवरीवन तो आज के कोर्स में हम कंप्लीट एसक्यूएल सीखने वाले हैं इसके साथ एक स्पेशल अनाउंसमेंट कि...

SQL Complete Revision Notes

1. Introduction to SQL

What is a Database?

- **Database** = Collection of organized data in digital format
- **DBMS** (Database Management System) = Software to manage databases (Create, Read, Update, Delete)
- **RDBMS** (Relational Database Management System) = Database with table-based structure

Types of Databases

Type	Description	Examples
Relational	Data in table format	MySQL, Oracle, SQL Server
Non-Relational (NoSQL)	Data without tables	MongoDB

What is SQL?

- **SQL** = Structured Query Language
- Used to interact with relational databases
- Performs **CRUD** operations: Create, Read, Update, Delete

2. SQL Command Types

Category	Commands	Purpose
DDL (Data Definition Language)	CREATE, DROP, ALTER, TRUNCATE	Define database structure
DQL (Data Query Language)	SELECT	Retrieve data
DML (Data Manipulation Language)	INSERT, UPDATE, DELETE	Manipulate data
DCL (Data Control Language)	GRANT, REVOKE	Permission control

Category	Commands	Purpose
TCL (Transaction Control)	COMMIT, ROLLBACK	Transaction management

3. MySQL Data Types

Numeric Types

Type	Range
INT	-2,147,483,648 to 2,147,483,647
TINYINT	-128 to 127
BIGINT	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
FLOAT	Decimal up to 23 digits
DOUBLE	Decimal up to 24 digits

String Types

Type	Description
CHAR(n)	Fixed length (0-255)
VARCHAR(n)	Variable length (0-65535)
TEXT	Large text data
BLOB	Binary large objects

Date Types

Type	Format
DATE	YYYY-MM-DD
TIME	HH:MM:SS
DATETIME	YYYY-MM-DD HH:MM:SS

4. Database Operations

sql

```
1  -- Create Database
2  CREATE DATABASE database_name;
3  CREATE DATABASE IF NOT EXISTS database_name;
4
5  -- Delete Database
6  DROP DATABASE database_name;
7  DROP DATABASE IF EXISTS database_name;
8
9  -- Show Databases
10 SHOW DATABASES;
11
12 -- Use Database
13 USE database_name;
14
15 -- Show Tables
16 SHOW TABLES;
```

5. Table Operations

Create Table

sql

```
1  CREATE TABLE table_name (
2      column1 datatype constraints,
3      column2 datatype constraints,
4      ...
5  );
6
7  -- Example
8  CREATE TABLE student (
9      id INT PRIMARY KEY,
10     name VARCHAR(50),
11     age INT NOT NULL,
12     city VARCHAR(30)
13 );
```

Insert Data

sql

```
1  -- Single row
2  INSERT INTO table_name (col1, col2, ...)
3  VALUES (val1, val2, ...);
```

```
4
5      -- Multiple rows
6      INSERT INTO table_name (col1, col2)
7      VALUES
8          (val1, val2),
9          (val3, val4);
```

Select Data

sql

```
1      -- All columns
2      SELECT * FROM table_name;
3
4      -- Specific columns
5      SELECT column1, column2 FROM table_name;
6
7      -- With DISTINCT (unique values)
8      SELECT DISTINCT column FROM table_name;
```

6. Constraints

Constraint	Description
NOT NULL	Column cannot be empty
UNIQUE	All values must be different
PRIMARY KEY	Unique + NOT NULL (only one per table)
FOREIGN KEY	References primary key of another table
DEFAULT	Default value if not specified
CHECK	Validates condition

sql

```
1      -- PRIMARY KEY
2      id INT PRIMARY KEY
3      -- OR
4      PRIMARY KEY (id)
5
6      -- FOREIGN KEY
7      FOREIGN KEY (col_name) REFERENCES other_table(col_name)
8
9      -- DEFAULT
```

```
10 salary INT DEFAULT 25000
11
12 -- CHECK
13 age INT CHECK (age >= 18)
```

7. WHERE Clause & Operators

Comparison Operators

sql

```
1 | =, !=, <>, >, <, >=, <=
```

Logical Operators

sql

```
1 | AND, OR, NOT
```

Special Operators

sql

```
1 -- BETWEEN (range)
2 WHERE age BETWEEN 18 AND 25
3
4 -- IN (list)
5 WHERE city IN ('Delhi', 'Mumbai', 'Pune')
6
7 -- NOT IN
8 WHERE city NOT IN ('Delhi', 'Mumbai')
9
10 -- LIKE (pattern matching)
11 WHERE name LIKE 'A%' -- Starts with A
12 WHERE name LIKE '%a' -- Ends with a
13 WHERE name LIKE '%am%' -- Contains 'am'
```

8. ORDER BY & LIMIT

sql

```
1  -- ORDER BY (ascending by default)
2  SELECT * FROM table_name ORDER BY column;
3  SELECT * FROM table_name ORDER BY column DESC;  --
   Descending
4
5  -- LIMIT (restrict rows)
6  SELECT * FROM table_name LIMIT 5;
7  SELECT * FROM table_name LIMIT 3 OFFSET 2;  -- Skip 2, then
   3 rows
8
9  -- Combined
10 SELECT * FROM table_name WHERE marks > 75 ORDER BY marks
    DESC LIMIT 3;
```

9. Aggregate Functions

Function	Description
COUNT()	Count rows
SUM()	Total sum
AVG()	Average value
MAX()	Maximum value
MIN()	Minimum value

sql

```
1  SELECT COUNT(*) FROM student;
2  SELECT MAX(marks) FROM student;
3  SELECT AVG(marks) FROM student;
4  SELECT SUM(salary) FROM employee;
```

10. GROUP BY & HAVING

GROUP BY

sql

```
1  -- Groups rows with same values
2  SELECT city, COUNT(*) FROM student GROUP BY city;
3  SELECT city, AVG(marks) FROM student GROUP BY city;
```

HAVING

sql

```
1  -- Filter groups (like WHERE but for groups)
2  SELECT city, COUNT(*)
3  FROM student
4  GROUP BY city
5  HAVING COUNT(*) > 2;
```

Order of Execution

sql

```
1  SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY
    → LIMIT
```

11. UPDATE & DELETE

sql

```
1  -- UPDATE
2  UPDATE table_name
3  SET column1 = value1, column2 = value2
4  WHERE condition;
5
6  UPDATE student
7  SET marks = marks + 10
8  WHERE marks < 80;
9
10 -- DELETE
11 DELETE FROM table_name WHERE condition;
12 DELETE FROM student WHERE marks < 33;
```

⚠ Warning: Don't forget WHERE in UPDATE/DELETE or all data will be affected!"

12. ALTER Command

sql

```
1  -- ADD column
2  ALTER TABLE table_name ADD column_name datatype;
3
4  -- DROP column
5  ALTER TABLE table_name DROP column_name;
6
7  -- MODIFY column (change datatype)
8  ALTER TABLE table_name MODIFY column_name new_datatype;
9
10 -- CHANGE column (rename)
11 ALTER TABLE table_name CHANGE old_name new_name datatype;
12
13 -- RENAME table
14 ALTER TABLE table_name RENAME TO new_name;
```

13. TRUNCATE vs DROP

Command	Description
DROP TABLE table_name	Delete table completely
TRUNCATE TABLE table_name	Delete all data, keep structure

14. Foreign Keys & Cascading

Foreign Key

sql

```
1  CREATE TABLE employee (
2      id INT PRIMARY KEY,
3      name VARCHAR(50),
4      dept_id INT,
5      FOREIGN KEY (dept_id) REFERENCES department(id)
6  );
```

Cascading

sql

```
1  -- ON DELETE CASCADE
2  -- ON UPDATE CASCADE
3  CREATE TABLE employee (
4      id INT PRIMARY KEY,
5      name VARCHAR(50),
6      dept_id INT,
7      FOREIGN KEY (dept_id) REFERENCES department(id)
8      ON DELETE CASCADE ON UPDATE CASCADE
9  );
```

- **CASCADE:** When parent table is updated/deleted, child table automatically updates/deletes

15. JOINS

Types of JOINS

JOIN Type	Description
INNER JOIN	Only matching rows from both tables
LEFT JOIN	All rows from left + matching from right
RIGHT JOIN	All rows from right + matching from left
FULL JOIN	All rows from both tables (MySQL: use UNION)
SELF JOIN	Join table with itself

Syntax

sql

```
1  -- INNER JOIN
2  SELECT * FROM table1 INNER JOIN table2
3  ON table1.col = table2.col;
4
5  -- LEFT JOIN
6  SELECT * FROM table1 LEFT JOIN table2
7  ON table1.col = table2.col;
8
9  -- RIGHT JOIN
10 SELECT * FROM table1 RIGHT JOIN table2
11 ON table1.col = table2.col;
12
```

```
13  -- SELF JOIN
14  SELECT * FROM employee e1
15  JOIN employee e2
16  ON e1.manager_id = e2.id;
```

16. UNION

sql

```
1  -- Combines results (removes duplicates)
2  SELECT column FROM table1
3  UNION
4  SELECT column FROM table2;
5
6  -- Includes duplicates
7  SELECT column FROM table1
8  UNION ALL
9  SELECT column FROM table2;
```

“Note: Number of columns and data types must be same”

17. Sub Queries

sql

```
1  -- In WHERE clause
2  SELECT name, marks FROM student
3  WHERE marks > (SELECT AVG(marks) FROM student);
4
5  -- In FROM clause (needs alias)
6  SELECT MAX(marks) FROM (
7      SELECT marks FROM student WHERE city = 'Delhi'
8  ) AS temp;
9
10 -- Using IN with subquery
11 SELECT name FROM student
12 WHERE roll_no IN (
13     SELECT roll_no FROM student WHERE marks > 90
14 );
```

18. Views

sql

```
1  -- Create View
2  CREATE VIEW view_name AS
3  SELECT column1, column2 FROM table_name WHERE condition;
4
5  -- Example
6  CREATE VIEW student_view AS
7  SELECT roll_no, name, marks FROM student;
8
9  -- Use View
10 SELECT * FROM student_view;
11
12 -- Drop View
13 DROP VIEW view_name;
```